Observables

In Observer pattern an observer subscribes to an Observable. Then that observer reacts to whatever item or sequence of items the Observable emits. This pattern facilitates concurrent operations because it does not need to block while waiting for the Observable to emit objects, but instead it creates a sentry in the form of an observer that stands ready to react appropriately at whatever future time the Observable does so.

When does an Observable begin emitting its sequence of items? It depends on the Observable. A "hot" Observable may begin emitting items as soon as it is created, and so any observer who later subscribes to that Observable may start observing the sequence somewhere in the middle. A "cold" Observable, on the other hand, waits until an observer subscribes to it before it begins to emit items, and so such an observer is guaranteed to see the whole sequence from the beginning.

Angular uses the reactiveX javascript library and its implementation of Observables for http service methods.

Wrapping an angular service in a promise or an observable is a good idea, since such a service can be called asynchronously and on completion can trigger further events (such as adding/modifying DOM elements).

Here we give an example of a service returning a promise for a result instead of the result itself.

This can then be handled appropriately at the component.

```
import { Injectable } from '@angular/core';
@Injectable()
export class MyService {
 data : any[];

serviceMethod(): Promise<any[]>
{
this.data=['a1','b2','c3']
return Promise.resolve(this.data);}}
```

Remember the stub method we created ? we have now defined it to return a promise of an object that can be used in our application.

The method now returns a promise of an array (any[] means array of any object types).

This can then be used by a component in the following manner.

```
this.myService.serviceMethod().then(/*code to process the response array*/);
```